



FREE SERVER KEEP-ALIVE GUIDE

Stop Your Free Backend From Sleeping Forever

Health Endpoint + UptimeRobot Setup Guide

1. The Problem

Free-tier hosting platforms like Render, Koyeb, and others spin down (put to sleep) your backend server after 15 minutes of inactivity. When a user visits your app after the server has slept, the first request takes 30–60 seconds to wake it up — causing a terrible user experience.

The Solution:

Add a lightweight health endpoint to your backend and use a free external monitoring service (UptimeRobot) to ping it every 5 minutes. This continuous traffic prevents the server from ever becoming "inactive", keeping it warm 24/7 at zero cost.

2. What Is a Health Endpoint?

A health endpoint is a minimal API route in your backend that returns a simple response indicating the server is running. It is used by monitoring tools, load balancers, and deployment platforms to check if your application is alive.

Requirements:

- Must be publicly accessible (no authentication required)
- Must be extremely lightweight (no database queries, no heavy computation)
- Must return HTTP 200 with a simple JSON response
- Must not expose secrets or sensitive application data
- Must not modify any application state

Expected Response:

```
GET /health

HTTP 200 OK
{
  "status": "ok"
}
```

3. Where to Add the Health Endpoint

Add the health endpoint in your main server/app file — the file where you initialize your web framework (Express, Flask, Django, FastAPI, etc.). Place it BEFORE your main application routes so it is not affected by authentication middleware, rate limiting, or other middleware.

Typical file locations by framework:

Framework	File to Edit	Placement
Express.js (Node.js)	app.js or server.js	Before app.use("/api", routes)

Flask (Python)	app.py	Before other <code>@app.route()</code>
Django (Python)	urls.py	In <code>urlpatterns</code> list
FastAPI (Python)	main.py	Before other routes
Spring Boot (Java)	Controller class	New <code>@GetMapping("/health")</code>
Go (net/http)	main.go	Before <code>http.ListenAndServe()</code>

4. Health Endpoint Code (Copy-Paste Ready)

Choose the code snippet that matches your backend framework. Add it to the file mentioned in Section 3.

Express.js / Node.js

JavaScript — app.js or server.js

```
// Add this BEFORE your main routes
// Example: Before app.use("/api", routes);

app.get("/health", (req, res) => {
  res.status(200).json({ status: "ok" });
});
```

Flask / Python

Python — app.py

```
# Add this in your main app.py file

@app.route("/health")
def health_check():
    return {"status": "ok"}, 200
```

FastAPI / Python

Python — main.py

```
# Add this in your main.py file

@app.get("/health")
def health_check():
    return {"status": "ok"}
```

Django / Python

Python — views.py + urls.py

```
# In views.py:
from django.http import JsonResponse

def health_check(request):
    return JsonResponse({"status": "ok"})

# In urls.py:
from .views import health_check

urlpatterns = [
    path("health/", health_check),
```

```
# ... your other URL patterns
]
```

Spring Boot / Java

Java — HealthController.java

```
// Create a new controller or add to existing one

@RestController
public class HealthController {

    @GetMapping("/health")
    public Map<String, String> healthCheck() {
        return Map.of("status", "ok");
    }
}
```

Go (net/http)

Go — main.go

```
// Add this handler in main.go

http.HandleFunc("/health", func(w http.ResponseWriter, r *http.Request) {
    w.Header().Set("Content-Type", "application/json")
    w.WriteHeader(http.StatusOK)
    w.Write([]byte(`{"status":"ok"}`))
})
```

5. Deploy the Health Endpoint

After adding the health endpoint code to your backend:

1. Save the file.
2. Commit and push to GitHub:

```
git add .
git commit -m "feat: add health endpoint for uptime monitoring"
git push origin main
```

3. Wait for your hosting platform (Render, Koyeb, etc.) to auto-deploy the new code.
4. Verify the endpoint is live by opening this URL in your browser:

<https://your-backend-url.onrender.com/health>

You should see:

```
{"status":"ok"}
```

6. UptimeRobot Setup (Free, 2 Minutes)

UptimeRobot is a free monitoring service that will send an HTTP request to your health endpoint every 5 minutes, 24/7. This continuous traffic prevents your server from ever sleeping.

Step 1: Create an Account

5. Go to <https://uptimerobot.com>
6. Click "Register for FREE" or sign in with GitHub.
7. Verify your email if prompted.

Step 2: Add a New Monitor

- On your UptimeRobot Dashboard, click the green "+ Add New Monitor" button (top-left).
- Fill in the following settings:

Setting	Value
Monitor Type	HTTP(s)
Friendly Name	Your Project Name (e.g., "My App Backend")
URL (or IP)	https://your-backend-url.onrender.com/health
Monitoring Interval	Every 5 minutes

Step 3: Configure Alert Contacts (Optional)

- UptimeRobot will ask if you want to receive notifications when your server goes down.
- Check the box next to your email if you want downtime alerts.
- This is optional but recommended.

Step 4: Create the Monitor

- Click "Create Monitor".
- Done! UptimeRobot will now ping your health endpoint every 5 minutes, 24/7.

7. Verify Everything Works

- Wait 5–10 minutes after creating the monitor.
- Go back to your UptimeRobot Dashboard.
- Your monitor should show a green "Up" badge with the response time.
- Click the monitor name to see response time graphs and uptime history.
- Visit your website/app — it should load instantly without any cold start delay!

8. How It Works (Behind the Scenes)

Every 5 minutes:

```
UptimeRobot (External Service)
|
| HTTP GET request
v
https://your-backend.onrender.com/health
|
| Returns: { "status": "ok" }
v
Server stays AWAKE because it received traffic
|
v
Render sees activity --> Does NOT spin down the server
```

Important Notes:

- This approach addresses inactivity-based spin-down only.
- It does NOT guarantee 100% uptime — server restarts, deployments, and platform maintenance can still cause brief downtime.
- This works on any free hosting platform that spins down after inactivity (Render, Koyeb, etc.).
- UptimeRobot's free plan supports up to 50 monitors with 5-minute intervals.

9. Quick Reference Checklist

- ☐ Add GET /health route to your backend (returns { "status": "ok" })
- ☐ Place it BEFORE authentication/rate-limiting middleware
- ☐ Commit and push to GitHub
- ☐ Wait for auto-deployment on your hosting platform
- ☐ Verify by visiting <https://your-backend-url/health> in browser
- ☐ Sign up at uptimerobot.com (free)
- ☐ Add New Monitor → HTTP(s) → paste your health endpoint URL
- ☐ Set monitoring interval to 5 minutes
- ☐ Create the monitor
- ☐ Verify green "Up" status on UptimeRobot dashboard after 5 minutes